Algebra and Discrete Mathematics
ADM_A_B

doc. Xiaolu Hou, PhD.

FIIT, STU
xiaolu.hou @ stuba.sk

Course Outline

- Vectors and matrices
- System of linear equations
- Matrix inverse and determinants
- Vector spaces, bases and matrix transformations
- Quiz one
- Matrix transformations, fundamental spaces and decompositions
- Eulerian tours and Hamiltonian cycles
- Shortest Paths, Project Scheduling, and Networks
- Quiz two
- Network Flows, Trees, and Matchings
- Make-up Quizzes
- Tutorial 12

Recommended reading

- Saoub, K. R. (2017). A tour through graph theory. Chapman and Hall/CRC.
  - Sections 5.1, 5.2, 5.3
  - Accessible online (free copy)
  - Alternative download link

Lecture outline

- Flow and capacity
- Rooted trees
- Bipartite graphs
- Matching terminology and strategies
- Augmenting paths and vertex covers
- Stable marriages
- Unacceptable partners

Network Flows, Trees, and Matchings

- Flow and capacity
- Rooted trees
- Bipartite graphs
- Matching terminology and strategies
- Augmenting paths and vertex covers
- Stable marriages
- Unacceptable partners

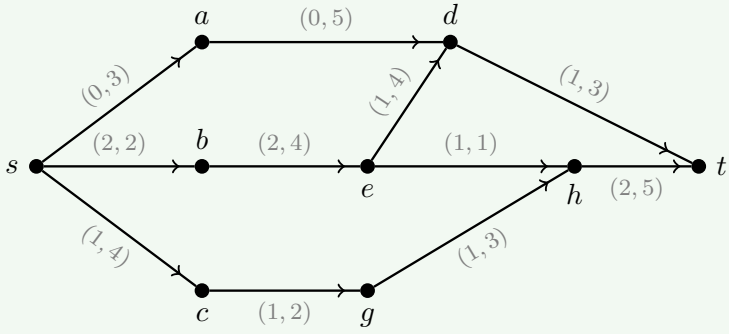
A different notion of a network

Definition

- A *network* is a connected digraph where each arc e has an associated nonnegative real number $c(e)$, called a *capacity*.
- The network has a designated starting vertex s, called the *source*, and a designated ending vertex t, called the *sink*.
- A *flow* f is a function that assigns a value $f(e)$ to each arc e of the network

Network – example

Example



- Each arc is given a two-part label, $(f(e), c(e))$
- The first component is the flow along the arc
- The second component is the capacity

Network

- The terms source and sink are motivated by the analogy of water flowing through a system of pipes, from the source to the sink.
- In this analogy, flow travels in the direction of the arcs, no arc can carry more than its capacity, and for every intermediate vertex the total inflow equals the total outflow.

Feasible flow

Definition

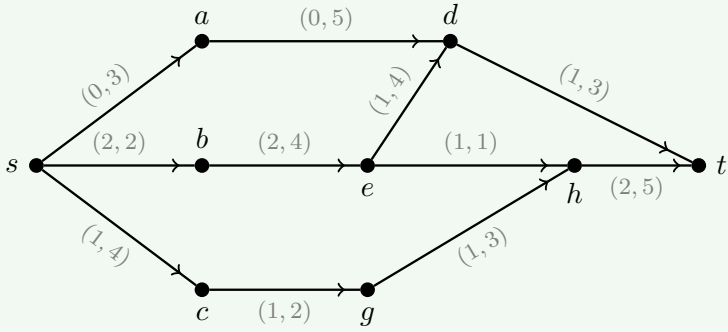
For a vertex v, let $f^-(v)$ represent the total flow entering v and $f^+(v)$ represent the total flow exiting v. A flow is *feasible* if it satisfies the following conditions

- $f(e) \geq 0$ for all arcs e
- $c(e) \geq f(e)$ for all arcs e
- $f^+(v) = f^-(v)$ for all vertices other than s and t
- $f^-(s) = f^+(t) = 0$

- The notation for inflow and outflow parallels that for in-degree and out-degree, except that here we sum the flow values on the arcs entering or leaving a vertex.
- The requirement that a flow is non-negative indicates the flow must travel in the direction of the arc, as a negative flow would indicate items going in the reverse direction
- The final condition is not necessary in theory, but more logical in practice and simplifies our analysis of flow problems

Network – example

Example



- $f^+(s) = 0 + 2 + 1 = 3,\ f^-(t) = 1 + 2 = 3$
- $c(ad) = 5,\ f(ad) = 0,\ c(ad) > f(ad)$
- $f^+(b) = f^-(b) = 2$

Maximum flow

Definition

- The *value* of a flow is defined as

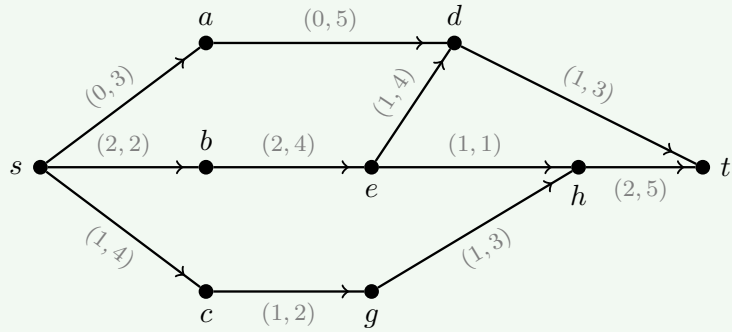
$$|f| = f^+(s) = f^-(t),$$

  i.e. the amount exiting the source which must also equal the flow entering the sink
- A *maximum flow* is a feasible flow of largest value

- In this course, we focus on networks with nonnegative integer capacities and consider only integer-valued flows.
- How can we find a maximum flow?

Network – example

Example



- Feasible flow, value 3
- Finding maximum flow is not as simple as putting every arc at capacity – e.g. if we had a flow of 5 along arc ad, flow along dt would be 6, more than its capacity

Slack

Definition

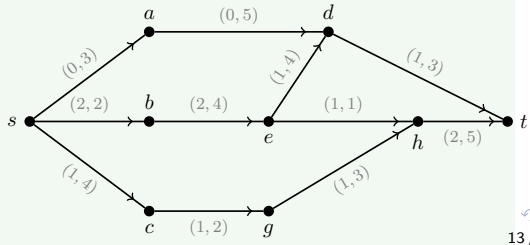
Let f be a flow along a network. The *slack* k of an arc is the difference between its capacity and flow

$$k(e) = c(e) - f(e).$$

Slack edge: $k(e) > 0$

Example

- $k(sa) = 3$, $k(sc) = 3$, $k(sb) = 0$
- We may want to increase flow along sa and sc but not sb

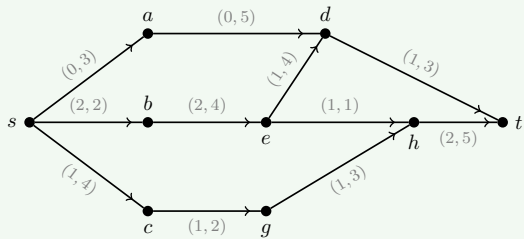


Chain

Definition

A *chain* K is a path in a digraph where the direction of the arcs are ignored

Example



- Both $sadt$ and $sadeht$ are chains
- $sadeht$ is not a directed path

Augmenting Flow Algorithm (Ford-Fulkerson Algorithm)

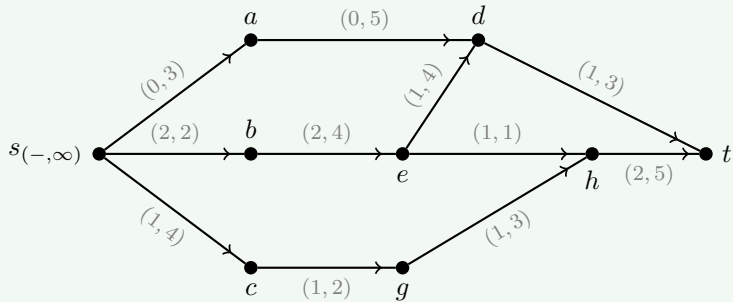
- This algorithm is similar in spirit to Dijkstra's Algorithm, which finds shortest paths in a graph or digraph.
- Vertices are assigned two-part labels that help us construct a chain along which the flow can be increased.
- The label consists of two parts, the second component is denoted by $\sigma(v)$
- Input: Network $G = (V, E, c)$, where each arc is given a capacity c, a designated source s and sink t, and an initial feasible flow f
- Output: Maximum flow f

Augmenting Flow Algorithm – steps

1. Label s with $(-, \infty)$, set $\sigma(v) = \infty$ for other vertices
2. Choose a labeled vertex x, scan x:
   a. For any arc yx, if $f(yx) > 0$ and y is unlabeled, then label y with $(x^-, \sigma(y))$, where $\sigma(y) = \min\{\sigma(x), f(yx)\}$
   b. For any arc xy, if $k(xy) > 0$ and y is unlabeled, then label y with $(x^+, \sigma(y))$, where $\sigma(y) = \min\{\sigma(x), k(xy)\}$
3. If t has been labeled, go to Step 4. Otherwise, choose another labeled vertex that has not yet been scanned and return to Step 2. If all labeled vertices have been scanned and t is still unlabeled, then f is a maximum flow.
4. Find an s-t chain K by backtracking from t to s using the vertex labels. Along the edges of K, increase the flow by $\sigma(t)$ units if they are in the forward direction and decrease by $\sigma(t)$ units in they are in the backward direction. Remove all vertex labels except that of s and return to Step 2

Augmenting Flow Algorithm – example

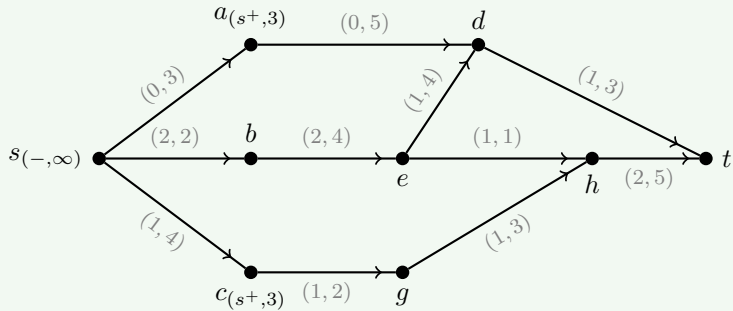
Example



- Step 1. Label s as $(-, \infty)$

Augmenting Flow Algorithm – example

Example

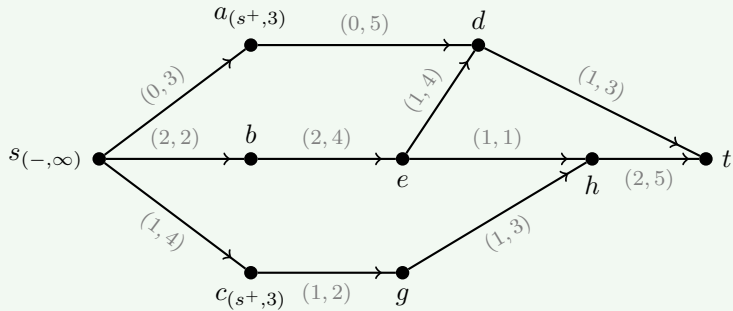


Step 2.

- There are no arcs to s.
- Arcs out of s: sa, sb, sc, with slack $3, 0, 3$
- Label a with $(s^+, 3)$, c with $(s^+, 3)$. Leave b unlabeled since sb has slack 0

Augmenting Flow Algorithm – example

Example

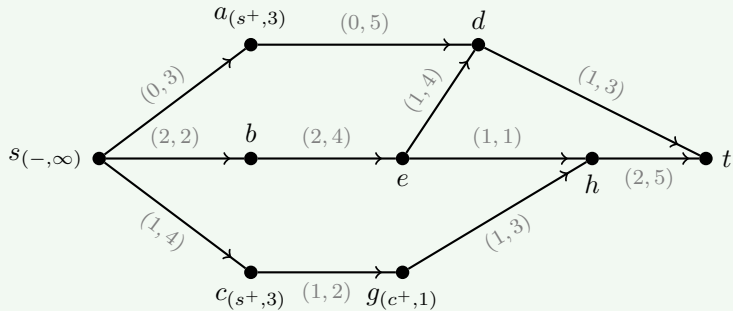


Step 3.

- Since t is not labeled, choose another labeled vertex that has not yet been scanned —either a or c.
- Choose c.

Augmenting Flow Algorithm – example

Example

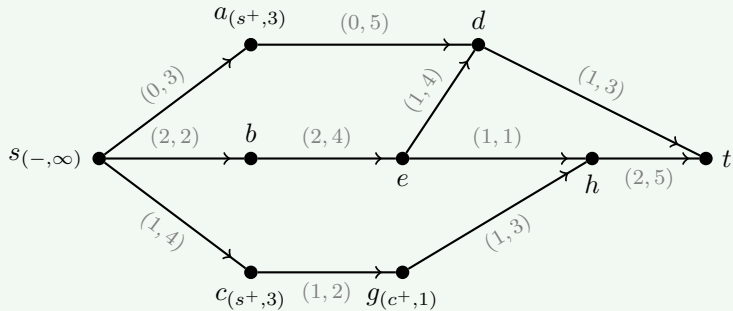


Step 2.

- The only arc entering c comes from a labeled vertex.
- Consider the arcs out of c – there is only cg with slack of 1
- Label g as $(c^+, 1)$, where $\sigma(g) = \min\{\sigma(c), k(cg)\} = \min\{3, 1\} = 1$

Augmenting Flow Algorithm – example

Example

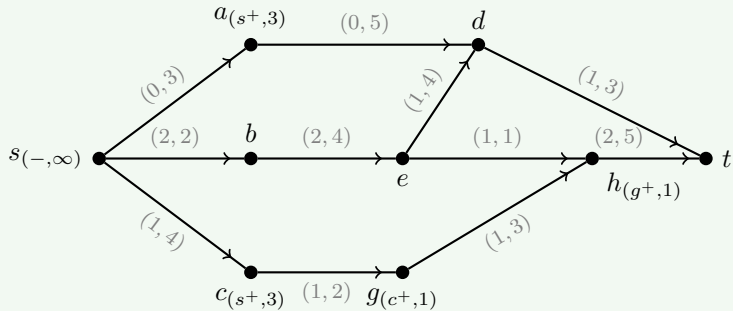


Step 3.

- t is not labeled
- We can scan either a or g
- Choose g

Augmenting Flow Algorithm – example

Example

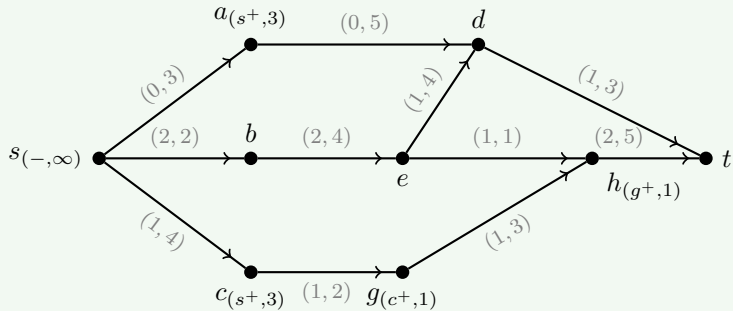


Step 2.

- The only arc entering g comes from a labeled vertex.
- The only arc out of g is gh
- Label h as $(g^+, 1)$, where $\sigma(h) = \min\{\sigma(g), k(gh)\} = \min\{1, 2\} = 1$

Augmenting Flow Algorithm – example

Example

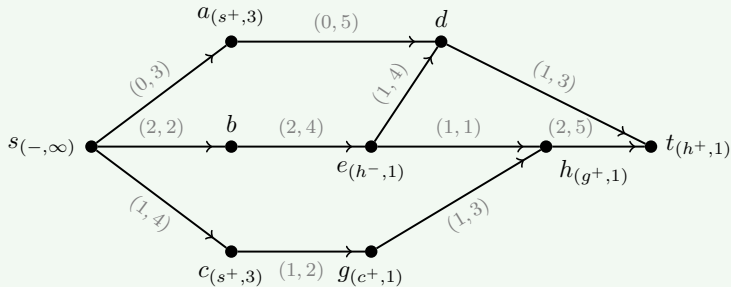


Step 3.

- t is not labeled, we can choose a or h
- Choose h

Augmenting Flow Algorithm – example

Example

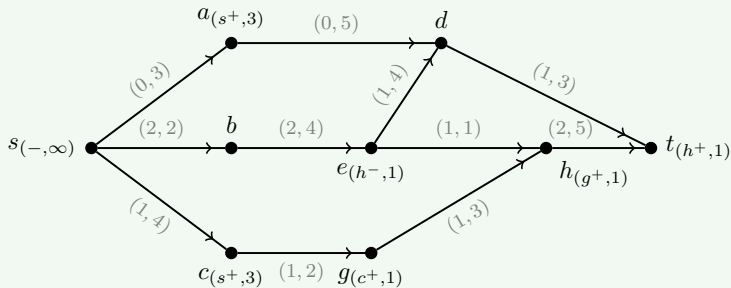


Step 2.

- There is one unlabeled vertex with an arc into h, namely e. Label e with $(h^-, 1)$, since $\sigma(e) = \min\{\sigma(h), f(eh)\} = \min\{1, 1\} = 1$.
- Label t as $(h^+, 1)$, where $\sigma(t) = \min\{\sigma(h), k(ht)\} = \min\{1, 3\} = 1$

Augmenting Flow Algorithm – example

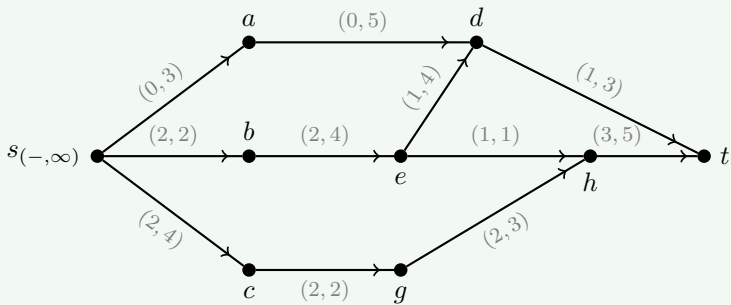
Example



- Step 3. t is labeled, go to Step 4
- Step 4. Backtracking from t to s using slack edges gives the chain $scght$.

Augmenting Flow Algorithm – example

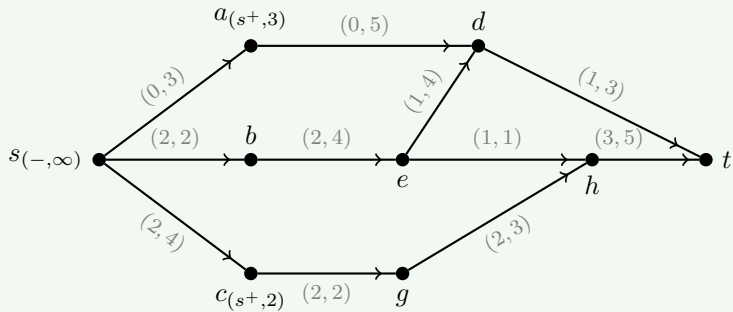
Example



- Step 4. Backtracking from t to s using slack edges gives the chain $scght$
  - Increase the flow by $\sigma(t) = 1$ units along each of these edges since all are in the forward direction.
  - Update the network flow and remove all labels except for s

Augmenting Flow Algorithm – example

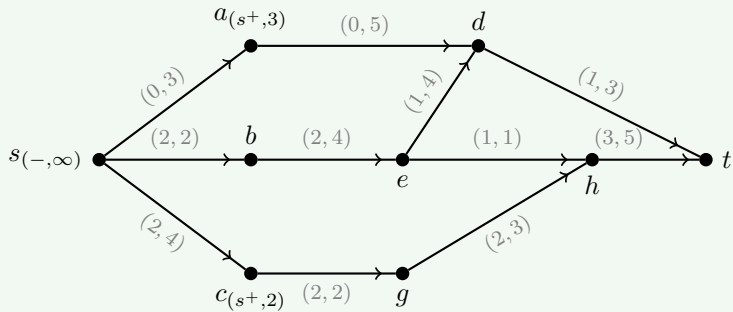
Example



- Step 2. Label a with $(s^+, 3)$ and c with $(s^+, 2)$
- Step 3. Choose c to scan

Augmenting Flow Algorithm – example

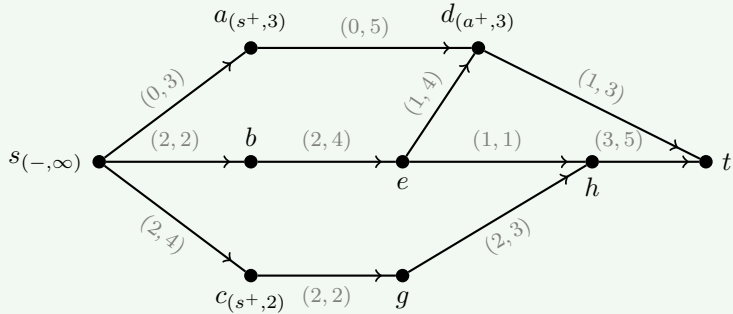
Example



- Step 2. No vertex to label, $k(cg) = 0$
- Step 3. Choose a to scan

Augmenting Flow Algorithm – example

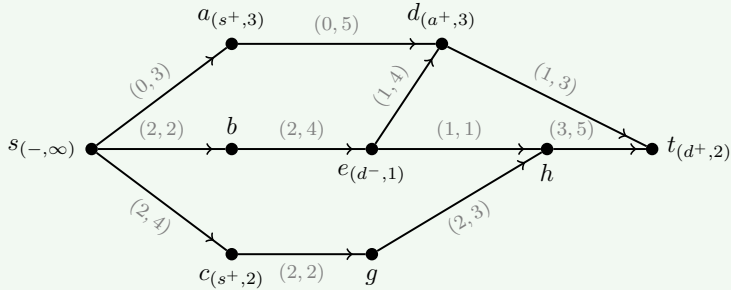
Example



- Step 2. Label d with $(a^+, 3)$, where $\sigma(d) = \min\{\sigma(a), k(ad)\} = \min\{3, 5\} = 3$
- Step 3. Only unscanned labeled vertex is d

Augmenting Flow Algorithm – example

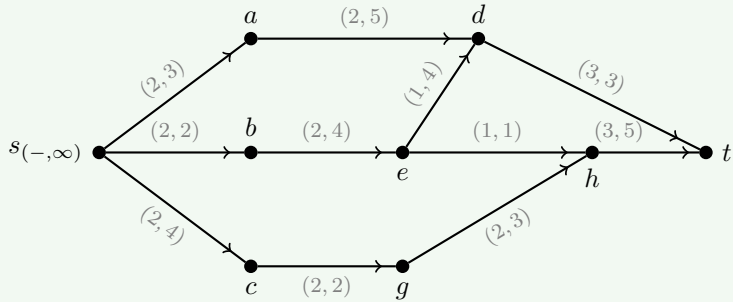
Example



- Step 2. Label e with $(d^-, 1)$, where $\sigma(e) = \min\{\sigma(d), f(ed)\} = \min\{3, 1\} = 1$. Label t with $(d^+, 2)$, $\sigma(t) = \min\{\sigma(d), k(dt)\} = \min\{3, 2\} = 2$
- Step 3. t is labeled, go to step 4
- Step 4. Backtracking gives the chain $sadt$

Augmenting Flow Algorithm – example

Example

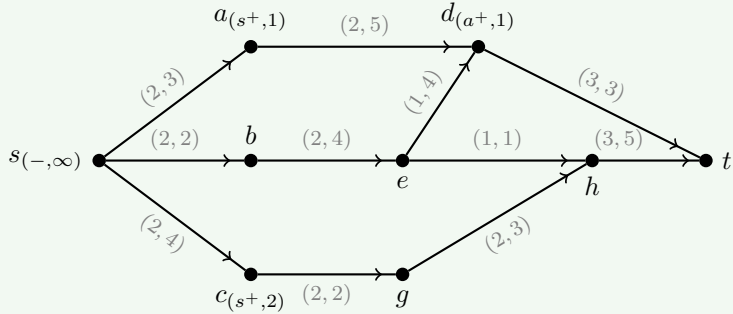


Step 4.

- Find chain $sadt$, increase the flow by $\sigma(t) = 2$ units along each of these edges since all are in the forward direction
- Update the network flow and remove all labels except for s

Augmenting Flow Algorithm – example

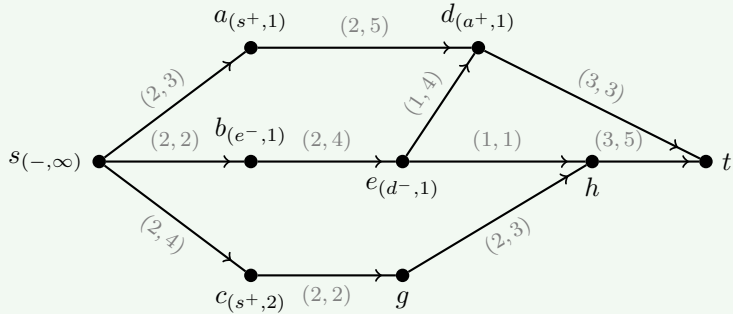
Example



- Step 2. Label a and c
- Step 3. Choose a
- Step 2. Label d
- Step 3. Choose d

Augmenting Flow Algorithm – example

Example



- Step 2. Label e as $(d^-, 1)$. Vertex t is not labeled because $k(dt) = 0$
- Step 3. Choose e
- Step 2. Label b
- Step 3. No vertices to be labeled, we get a maximum flow

Augmenting Flow Algorithm

- When the Augmenting Flow Algorithm halts, a maximum flow has been achieved, though understanding why this flow is indeed maximum requires additional terminology and results

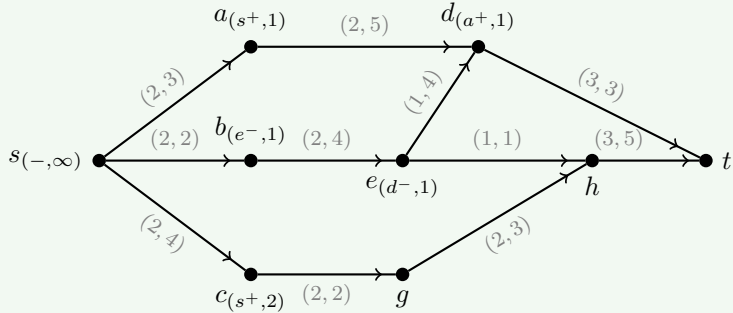
Cut

Definition

- Let P be a set of vertices and $\overline{P} = V - P$.
- A *cut* $(P, \overline{P})$ is the set of all arcs xy where $x \in P$ and $y \in \overline{P}$
- An $s - t$ *cut* is a cut in which $s \in P$, $t \in \overline{P}$

Cut – example

Example



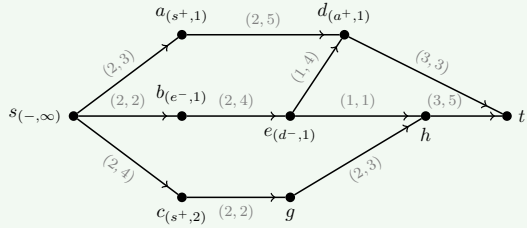
- $P = \{s, a, e, g\}$, $\overline{P} = \{b, c, d, h, t\}$
- $(P, \overline{P}) = \{sb, sc, ad, ed, eh, gh\}$
- Note: $be \notin (P, \overline{P})$ because $b \in \overline{P}$, $e \in P$

Capacity

Definition

The *capacity* of a cut, $c(P, \overline{P})$, is defined as the sum of the capacities of the arcs that comprise the cut.

Example



- $P = \{s, a, e, g\}$, $\overline{P} = \{b, c, d, h, t\}$, $(P, \overline{P}) = \{sb, sc, ad, ed, eh, gh\}$, $c(P, \overline{P}) = 2 + 4 + 5 + 4 + 1 + 3 = 19$
- $P = \{s\}$, $c(P, \overline{P}) = 9$

Max Flow-Min Cut

Theorem

In any directed network, the value of a maximum $s-t$ flow equals the capacity of a minimum $s-t$ cut.

- The difficulty in using this theorem directly is that one must still identify a minimum cut.
- We can use the vertex labeling procedure to obtain our minimum cut.

Min-Cut Method

Steps

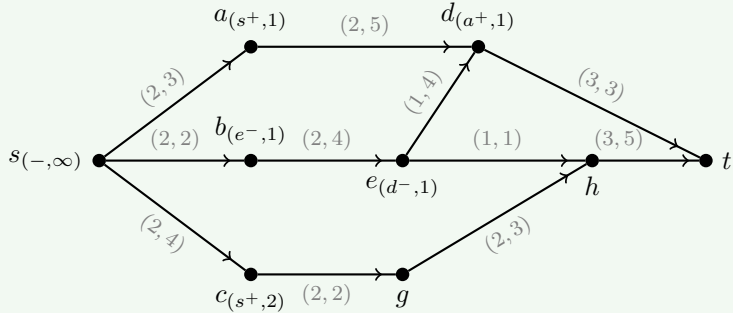
1. Let $G = (V, A, c)$ be a network with a designated source s and sink t and each arc is given a capacity c
2. Apply the Augmenting Flow Algorithm
3. Define an $s - t$ cut $(P, \overline{P})$ where P is the set of labeled vertices from the final implementation of the algorithm
4. $(P, \overline{P})$ is a minimum $s - t$ cut for G

Note

In practice, we can perform the Augmenting Flow Algorithm and the Min-Cut Method simultaneously, thus finding a maximum flow and providing a proof that it is maximum (through the use of a minimum cut) in one complete procedure.

Min-Cut Method – example

Example



- $P = \{s, a, b, c, d, e\}$, $\overline{P} = \{g, h, t\}$
- $(P, \overline{P}) = \{dt, eh, cg\}$
- $c(P, \overline{P}) = 3 + 1 + 2 = 6$

Network Flows, Trees, and Matchings

- Flow and capacity
- Rooted trees
- Bipartite graphs
- Matching terminology and strategies
- Augmenting paths and vertex covers
- Stable marriages
- Unacceptable partners

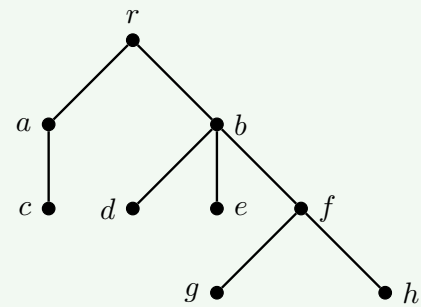
Definition

Definition

- A *rooted tree* is a tree T with a special designated vertex r, called the *root*
- The *level* of any vertex in T is defined as the length of its shortest path to r
- The *height* of a rooted tree is the largest level for any vertex in T

Example

- root r: level 0
- a, b: level 1
- c, d, e, f: level 2
- g, h: level 3
- height of the tree: 3



Terminologies

Definition

Let T be a tree with root r. Then for any vertices x and y

- $y \neq x$ is on the unique path from x to r: x is a *descendant* of y; y is an *ancestor* of x
- x is a descendant of y and exactly one level below y: x is a *child* of y, y is a *parent* of x
- x is a *sibling* of y if $x \neq y$ and x and y have the same parent.

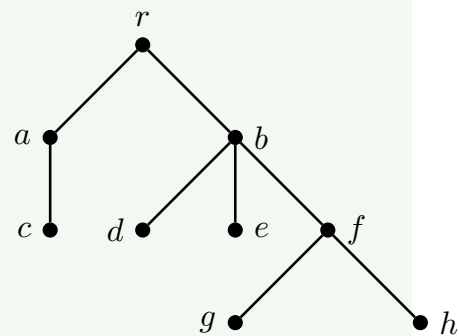
Note

Analogy: family tree

Terminologies – example

Example

- parent of a: r
- child of a: c
- parent of e: b
- e has no children
- ancestors of g: f, b, r – unique path from g to r is $gfbr$
- descendants of b: d, e, f, g, h
- siblings of d: e, f



Depth-First Search Tree

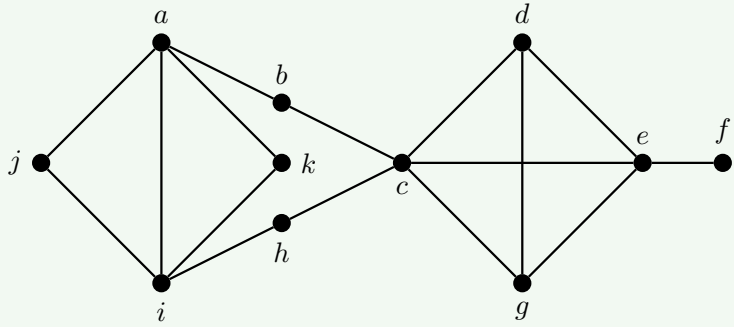
- The main idea is to follow a path as far as possible from the root before backtracking.
- If this path does not include all vertices, we backtrack and build further branches to obtain a tree.
- The formal description of the algorithm uses an ordered list of neighbors for each vertex, and this order determines which new vertex is added next.
- For simplicity, we will always use an alphabetical order when considering neighbor lists
- Input: Simple (no multi-edges or loops) connected graph $G = (V, E)$ and a designated root vertex r
- Output: Depth-first search tree T

Depth-First Search Tree – steps

1. Initialize the DFS tree $T = (V', E')$ with $V' = \{r\}$ and $E' = \emptyset$. Set r as the current vertex v.
2. Select the first unvisited neighbor x of the current vertex v. Add vertex x and edge vx to T, and recursively repeat Step 2 with x as the new current vertex until no unvisited neighbors remain.
3. If all vertices of G are now in T, then T is the depth-first search tree. Otherwise, backtrack along the path from the last visited vertex to the root until you find a vertex v with an unvisited neighbor. Use v as the current vertex and repeat Step 2.

Depth-First Search Tree – example

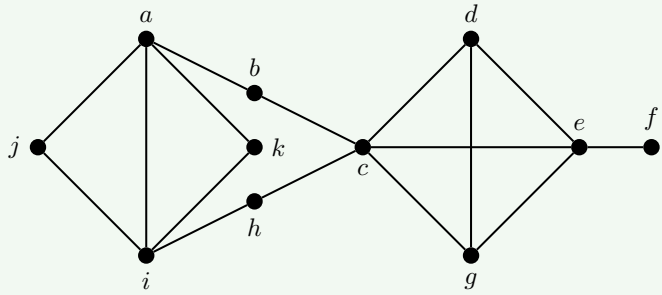
Example



- Suppose a is the root
- Step 1. Current vertex is a
- Step 2. Add b, c, d, e, f in order. This process stops at f, since f has no unvisited neighbors in G.

Depth-First Search Tree – example

Example

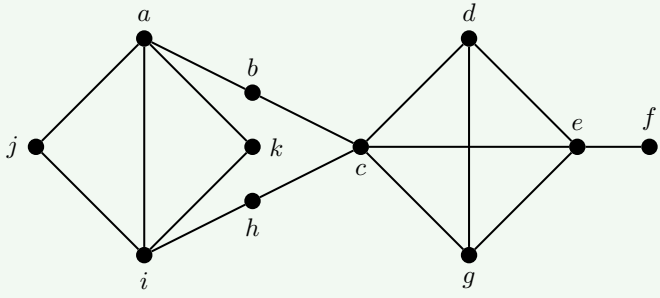


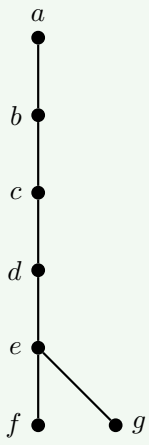


- Step 3. Backtracking along the path, the first vertex with an unvisited neighbor is e
- Step 2. Add edge eg and vertex g to T

Depth-First Search Tree – example

Example

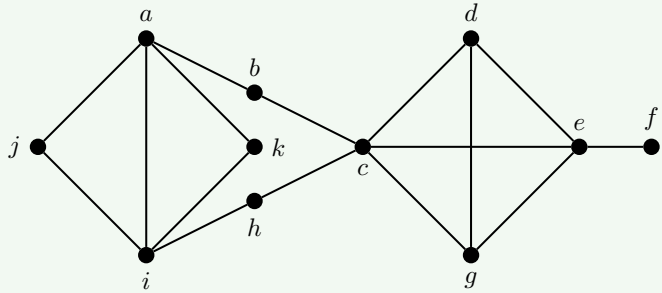


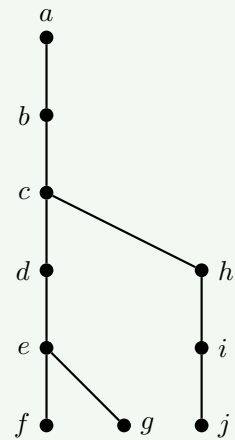


- Step 3. Backtracking along the path from g to a, the first vertex with an unvisited neighbor is c
- Step 2. Add h, i, j in order.

Depth-First Search Tree – example

Example

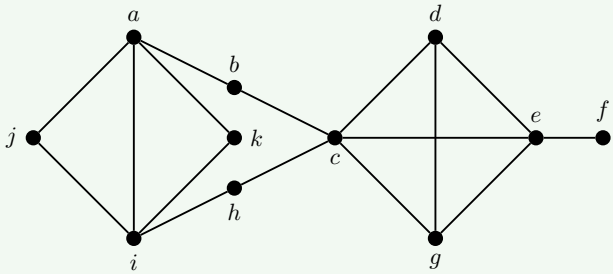


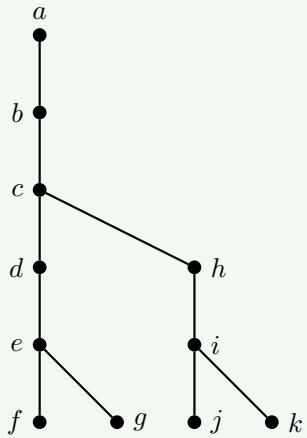


- Step 3. Backtracking along the path from j to a, the first vertex with an unvisited neighbor is i
- Step 2. Add k

Depth-First Search Tree – example

Example





- T contains all the vertices of G
- The resulting tree is the depth-first search tree
- The tree has height 5, with one vertex at each of levels 1 and 2, two vertices at each of levels 3 and 4, and four vertices at level 5.

Remark

- If the graph is not connected, we can slightly adjust the algorithm and get a forest as the output
- Search for unvisited vertices when no vertex with unvisited neighbors can be found

Breadth-First Search Tree

- Main objective is to add as many neighbors of the root as possible in the first step
- At each subsequent step, we add all unvisited neighbors of the most recently added vertices.
- As with depth-first search, we will use alphabetical ordering of neighbor lists.
- Input: Simple connected graph $G = (V, E)$ and a designated root vertex r
- Output: Breadth-first tree T

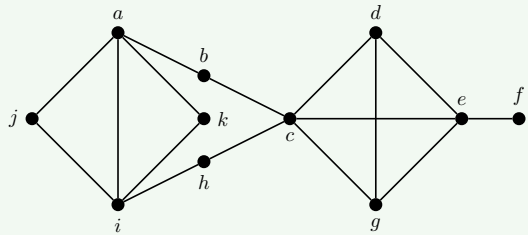
Breadth-First Search Tree – steps

1. Initialize the BFS tree $T = (V', E')$ with the root vertex r, i.e., $V' = \{r\}$, and mark r as visited.
2. Add all neighbors of r to V', and add the corresponding edges from r to each neighbor to E'. Mark all these neighbors as visited. Let this set of newly added vertices be the current level.
3. For each vertex v in the current level (in alphabetical order):
   - Add all unvisited neighbors x of v to V'.
   - Add the edge (v, x) to E'.
   - Mark each such neighbor x as visited.

   Let the collection of all such newly added vertices form the next level.
4. If T now includes all vertices of G, the process is complete. Otherwise, repeat step 3 using the next level as the current level.

Breadth-First Search Tree – example

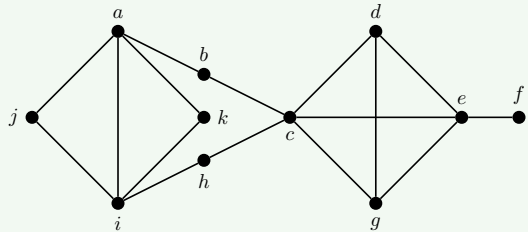
Example



- Let's consider the same example
- Take a as the root
- Step 1. Add a, b, i, j, k
- Step 2. Current level: b, i, j, k

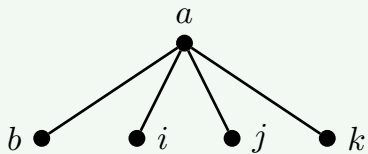
Breadth-First Search Tree – example

Example



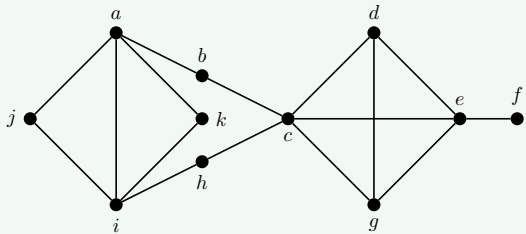
Step 3

- Vertex b: add neighbor c, and edge bc
- Vertex i: add neighbor h and edge ih
- j and k do not have any unvisited neighbors

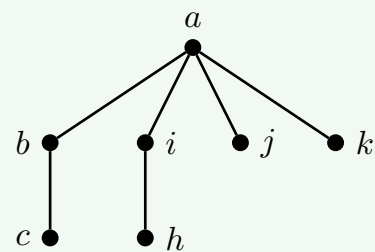


Breadth-First Search Tree – example

Example

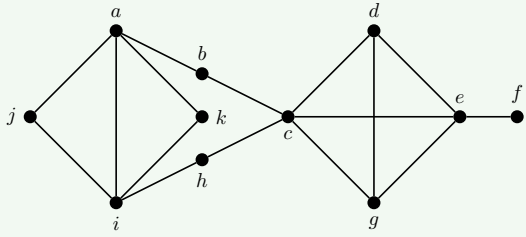


- Step 4. T does not contain all vertices, repeat step 3. Current level: c, h
- Step 3.
  - Vertex c: unvisited neighbors d, e, g
  - Vertex h has no unvisited neighbors

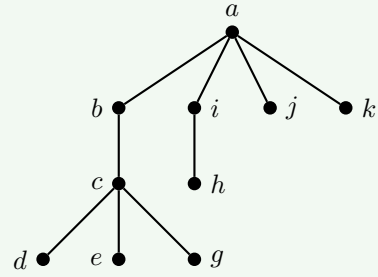


Breadth-First Search Tree – example

Example

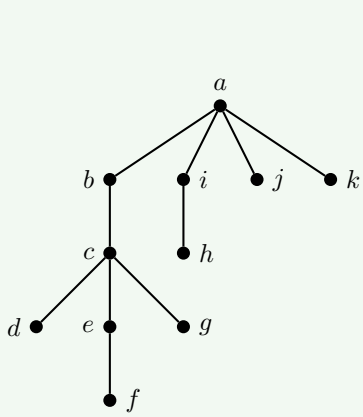


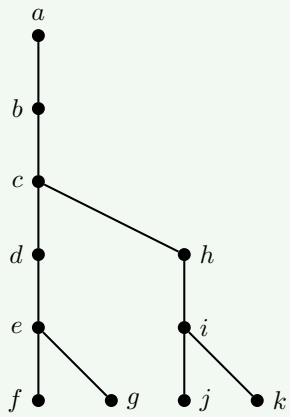
- Step 4. T does not contain all vertices, repeat step 3. Current level: d, e, g
- Step 3.
  - Vertex d has no unvisited neighbors
  - Vertex e: add neighbor f and edge ef
  - Now we have all vertices



Breadth-First Search Tree – example

Example

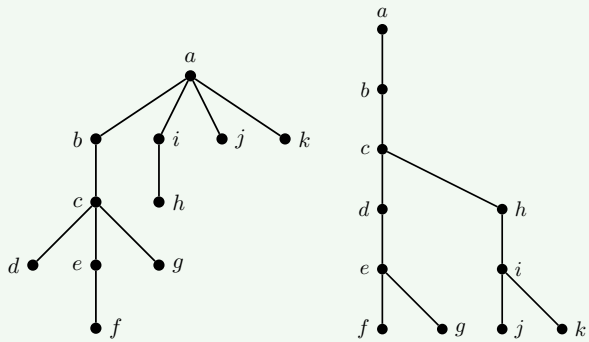




- Left: BFS tree; Right: DFS tree
- BFS trees are often shorter in height than their DFS counterparts.

Breadth-First Search Tree – example

Example



- The BFS tree has height 4, with four vertices on level 1, two on level 2, three on level 3, and one on level 4.
- The DFS tree has height 5, with one vertex at each of levels 1 and 2, two vertices at each of levels 3 and 4, and four vertices at level 5.

Network Flows, Trees, and Matchings

- Flow and capacity
- Rooted trees
- Bipartite graphs
- Matching terminology and strategies
- Augmenting paths and vertex covers
- Stable marriages
- Unacceptable partners

Bipartite graphs

Definition

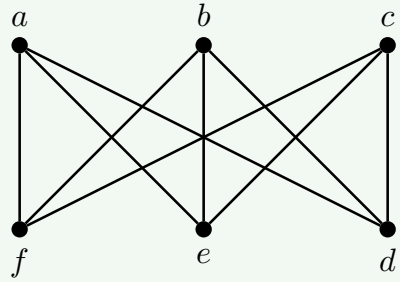
A graph $G = (V, E)$ is *bipartite* if the vertices can be partitioned into two sets, X and Y, so that

$$X \cap Y = \emptyset, \quad V = X \cup Y,$$

and every edge has exactly one endpoint in X and the other endpoint in Y.

Example

- $X = \{a, b, c\}$
- $Y = \{d, e, f\}$



Different drawing

- The drawing of a graph can vary
- It may not be easy to see if the graph is bipartite
- But, if you are traveling along a path or cycle in a graph, the vertices will need to alternate between the two parts of the vertex set, such as a vertex from X, then Y, then X, etc
- So if a cycle exists in the graph, it must have even length since otherwise two vertices along the cycle would come from the same part

Determine if a graph is bipartite

Theorem

A graph G is bipartite iff there are no odd cycles in G.

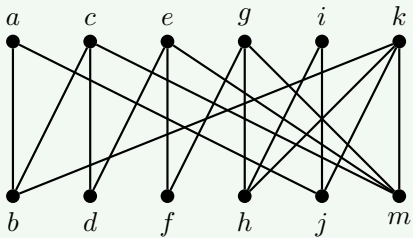
Note

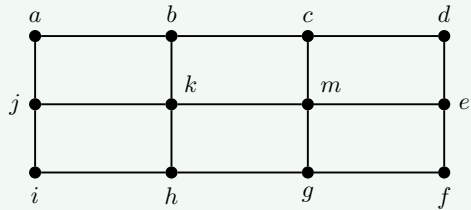
- In practice, we usually search for odd cycles within a graph and if we cannot find any, we try to redraw the graph to emphasize that it is bipartite
- A bipartite graph can have multi-edges (and so need not be simple) but cannot have loops (since these are odd cycles of length 1)

Determine if a graph is bipartite – example

Example

- The graph is bipartite
- $X = \{a, c, e, g, i, k\}$,
  $Y = \{b, d, f, h, j, m\}$

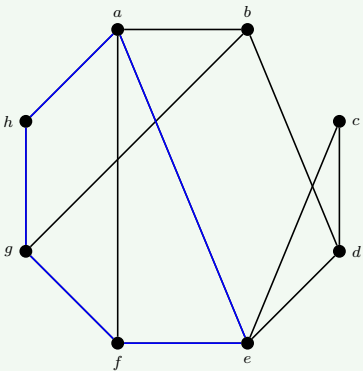


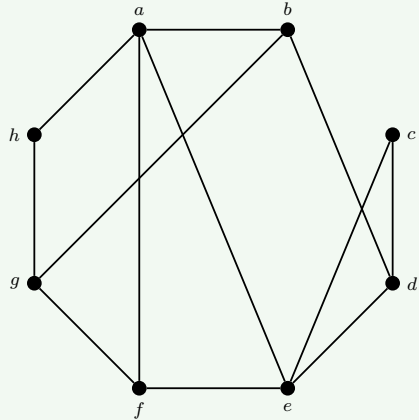


Determine if a graph is bipartite – example

Example

- Not bipartite since there are odd cycles, such as the 5-cycle $aefgha$





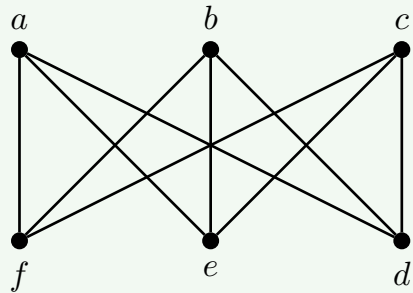
Complete bipartite graph

Definition

A simple bipartite graph $G = (X \cup Y, E)$ is a *complete bipartite graph* if every vertex in X is adjacent to every vertex in Y. If $|X| = m$, $|Y| = n$, we write $K_{m,n}$

Example

$K_{3,3}$



Network Flows, Trees, and Matchings

- Flow and capacity
- Rooted trees
- Bipartite graphs
- Matching terminology and strategies
- Augmenting paths and vertex covers
- Stable marriages
- Unacceptable partners

Definitions

Definition

Given a graph $G = (V, E)$

- Two edges that do not share an endpoint are called *independent* edges
- A *matching* M is a subset of E such that every two distinct edges in M are independent
- The size of a matching, denoted $|M|$, is the number of edges in the matching

Applications

- Marriages
- Assign tasks to employees
  - None of the tasks rely on each other but no person has enough time to complete more than one task.
  - In addition, most employees are only qualified to complete some of the tasks

Remark

Matching problems often (though not always) make use of bipartite graphs since the items being matched are usually of two distinct types

Matching – example

Example

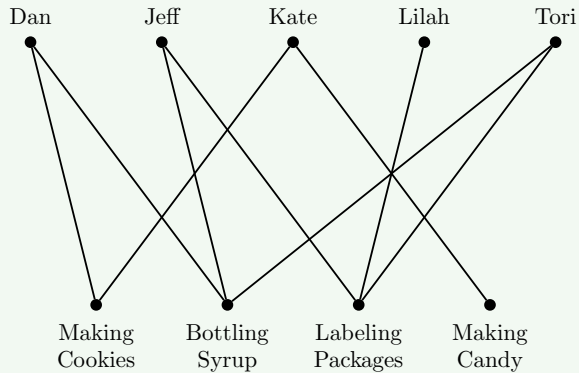
- Each employee is shown along with the jobs for which he or she is qualified
- Model using a bipartite graph where X consists of the employees and Y consists of the tasks

Employee	Task	
Dan	Making Cookies	Bottling Syrup
Jeff	Labeling Packages	Bottling Syrup
Kate	Making Candy	Making Cookies
Lilah	Labeling Packages	
Tori	Labeling Packages	Bottling Syrup

Matching – example

Example

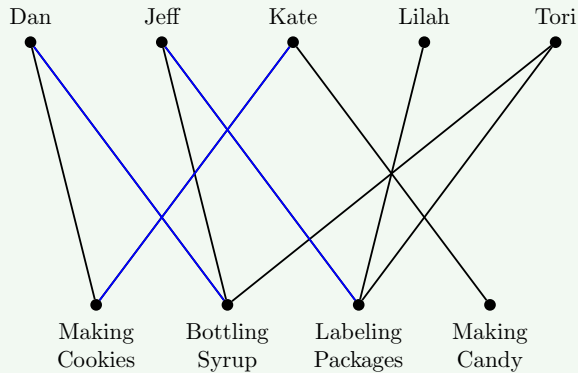
- X consists of the employees and Y consists of the tasks
- We draw an edge between two vertices a and b if employee a is capable of completing the task b



Matching – example

Example

- One possible matching



Matching problem

- What is the important criterion for a solution, and how does that translate to a matching?
- Is it more important for each employee to have a task or for every task to be completed

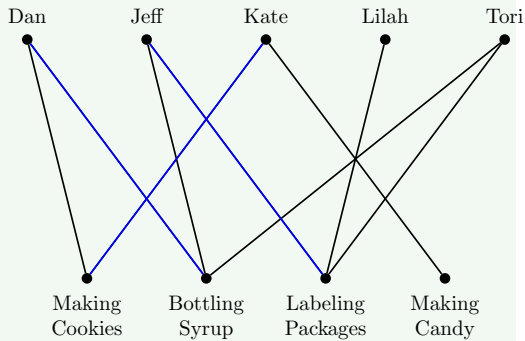
Saturated vertices

Definition

A vertex is *saturated* by a matching M if it is incident to an edge of the matching; otherwise, it is *unsaturated*.

Example

- Saturated vertices: Dan, Jeff, Kate, Making Cookies, Bottling Syrup, and Labeling Packages
- Unsaturated vertices: Making candy, Lilah, and Tori



Different kinds of matchings

Definition

Given a matching M on a graph G, M is

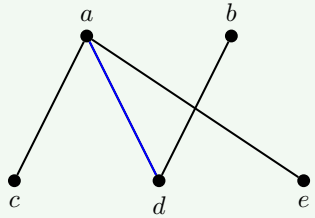
- *maximal* if M cannot be enlarged by adding an edge
- *maximum* if M is of the largest size among all possible matchings
- *perfect* if M saturates every vertex of G
- an X-matching if it saturates every vertex from the collection of vertices X

Note

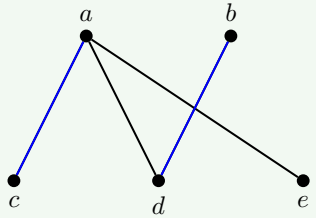
- A perfect matching is automatically maximum and a maximum matching is automatically maximal, though the reverse need not be true
- There can be several matchings of equal size

Maximal and maximum matching – example

Example



Maximal matching



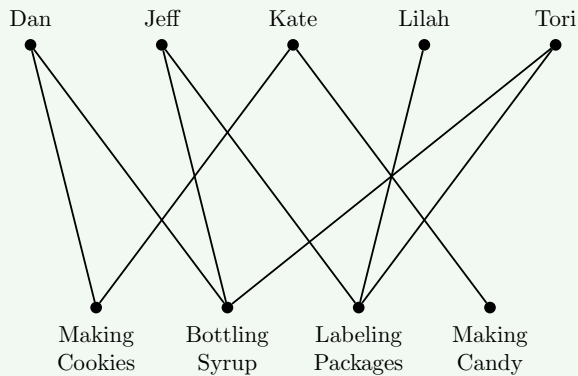
Maximum matching

- Maximal: no other edges in the graph can be added since the remaining edges require the use of a saturated vertex (either a or d)
- Maximum: There is no way for a matching to contain three edges - otherwise a would have two edges incident to it
- Matching on the right is an X-matching for $X = \{a, b\}$
- Neither matching is perfect since not every vertex is saturated

Find a matching – example

Example

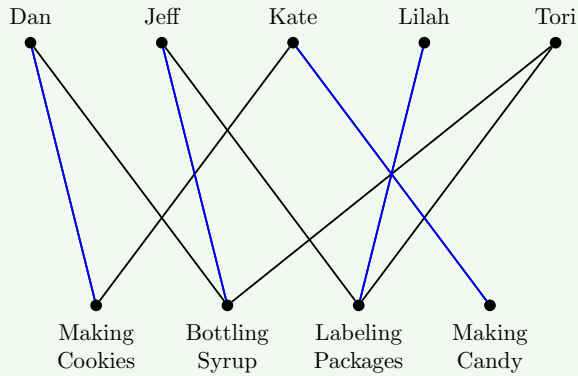
- We need the tasks to be completed but do not need every employee to be assigned a task
- We need an X-matching, where X consists of the vertices representing the tasks



Find a matching – example

Example

- One possible matching
- All tasks are assigned to an employee, but not all employees have a task



Hall's Marriage Theorem

Theorem (Hall's Marriage Theorem)

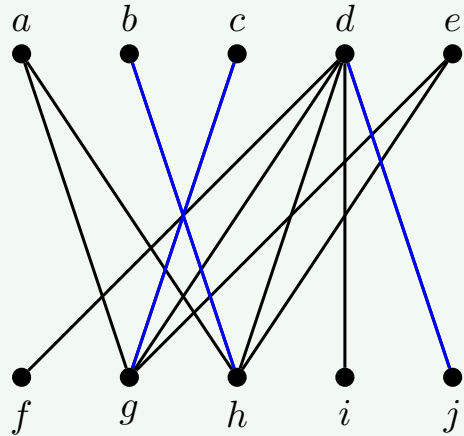
Given a bipartite graph $G = (X \cup Y, E)$, there exists an X-matching if and only if every subset $S \subseteq X$ satisfies $|S| \leq |N(S)|$.

- The neighbor set of a set of vertices S, $N(S)$, consists of all vertices incident to at least one vertex from S
- Hall's Marriage Theorem does not give a definitive answer about the size of a maximum matching but rather gives us the tools to reason why an X-matching does not exist

Hall's Marriage Theorem – example

Example

- Let $Y = \{f, g, h, i, j\}$, and take $S = \{f, i, j\} \subseteq Y$.
- $N(S) = \{d\}$
- By Hall's Marriage Theorem, there is no Y-matching.
- Since at most one of the vertices from S can be paired with d, we know the maximum matching can contain at most 3 edges
- The matching on the right is maximum



Find a maximum matching

- Next, we use a specific type of path and a collection of vertices to answer the optimization question: how do we find a maximum matching?

Network Flows, Trees, and Matchings

- Flow and capacity
- Rooted trees
- Bipartite graphs
- Matching terminology and strategies
- Augmenting paths and vertex covers
- Stable marriages
- Unacceptable partners

Alternating and augmenting paths

Definition

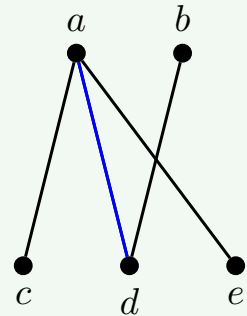
Given a matching M of a graph G, a path is called

- M-*alternating* if the edges in the path alternate between edges that are in M and edges that are not in M
- M-*augmenting* if it is M-alternating and both endpoints of the path are unsaturated by M, implying both the starting and ending edges of the path are not in M

Alternating and augmenting paths – example

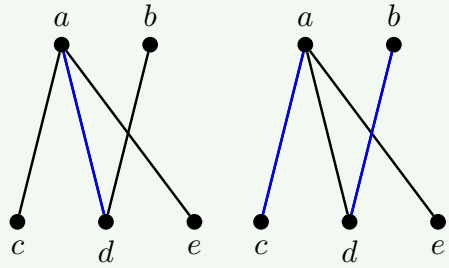
Example

- $cadb$ is alternating and augmenting (both c and b are unsaturated)
- If we switch the edges along this path we get a larger matching
- This switching procedure removes the matched edges and adds the previously unmatched edges along an augmenting path
- Since the path is augmenting, the matching increases in size by one edge



Alternating and augmenting paths – example

Example



- This switching procedure removes the matched edges and adds the previously unmatched edges along an augmenting path
- Since the path is augmenting, the matching increases in size by one edge
- $cadb$ is still alternating, but not augmenting

Berge's Theorem

Theorem (Berge's Theorem)

A matching M of a graph G is maximum iff G does not contain any M-augmenting paths.

Note

Unlike Hall's Theorem, Berge's Theorem holds for both bipartite graphs and non-bipartite graphs.

Augmenting Path Algorithm

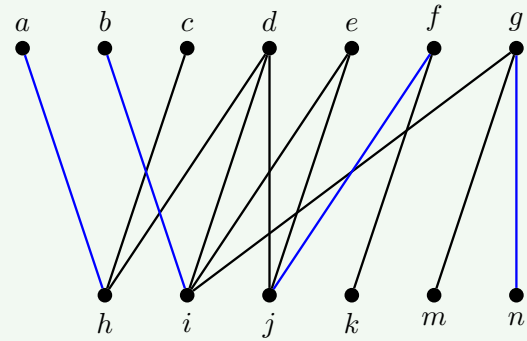
- Input: Bipartite graph $G = (X \cup Y, E)$
- Output: Maximum matching for G

Augmenting Path Algorithm – steps

1. Find an arbitrary matching M
2. $U =$ set of unsaturated vertices in X
3. If $U = \emptyset$, M is a maximum matching; otherwise, select a vertex $x \in U$
4. Consider $y \in N(x)$, if y is unsaturated, go to step 5, otherwise, step 6
5. Add the edge xy to M to obtain a larger matching M'. Return to Step 2 and recompute U.
6. Search for an M-augmenting path starting at x by exploring all possible M-alternating paths from x.
   a. If an M-augmenting path is found, switch the matched and unmatched edges along this path to obtain a larger matching M'. Set $M = M'$, return to Step 2, and recompute U.
   b. If no M-augmenting path starting at x exists, return to step 4 and continue with another unsaturated vertex in U.
7. Repeat steps 2 - 4 until all vertices from U have been considered

Augmenting Path Algorithm – example

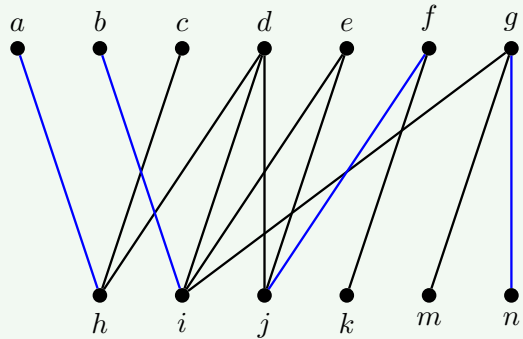
Example



- Step 1. arbitrary matching
- Step 2. $U = \{c, d, e\}$

Augmenting Path Algorithm – example

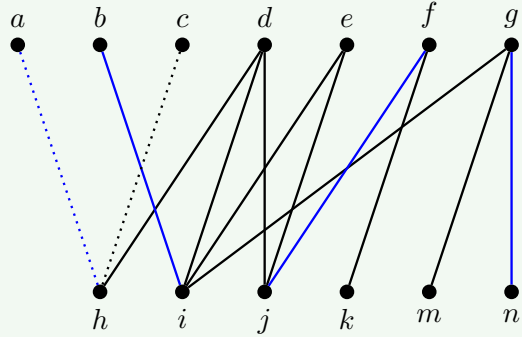
Example



- Step 3. select vertex $c \in U$
- Step 4. only neighbor of c is h, which is saturated
- Step 6. form an M-alternating path starting with edge ch, cha

Augmenting Path Algorithm – example

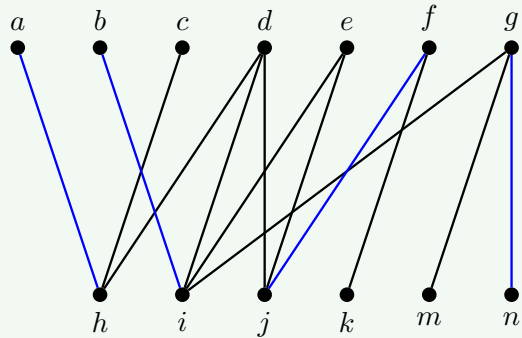
Example



- Step 6. cha is not augmenting, go to step 4
- Step 4. no other neighbor to choose, go to step 3

Augmenting Path Algorithm – example

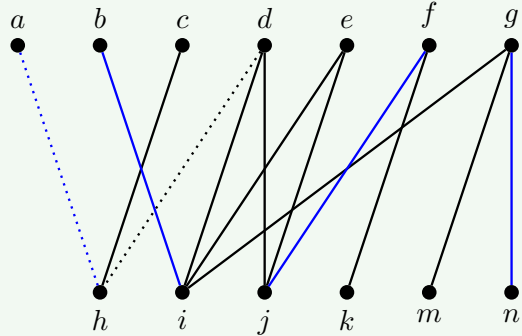
Example



- $U = \{c, d, e\}$
- Step 3. select vertex $d \in U$
- Step 4. $N(d) = \{h, i, j\}$, consider h

Augmenting Path Algorithm – example

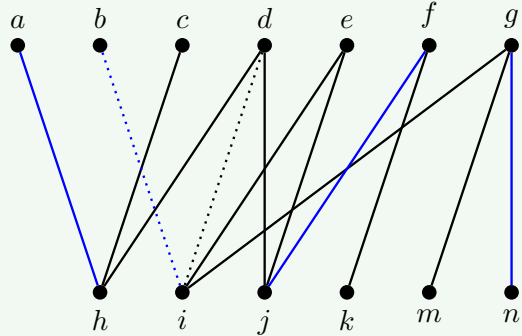
Example



- Step 6. dha is not M-augmenting, go to step 4
- Step 4. $N(d) = \{h, i, j\}$, consider i

Augmenting Path Algorithm – example

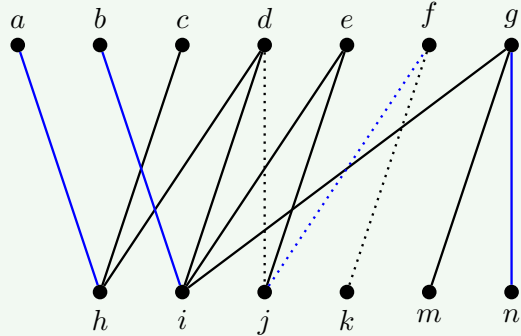
Example



- Step 6. dib is not M-augmenting, go to step 4
- Step 4. $N(d) = \{h, i, j\}$, consider j

Augmenting Path Algorithm – example

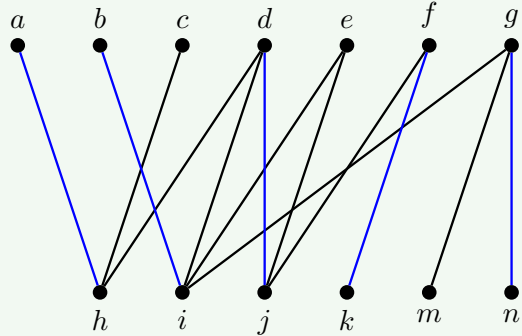
Example



- Step 6. $djfk$ is M-augmenting, so form a new matching by removing fj and adding dj and fk.

Augmenting Path Algorithm – example

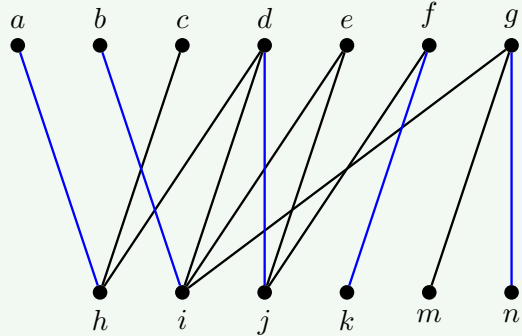
Example



- Step 2. $U = \{c, e\}$
- Step 3. select $c \in U$
- Step 4. only neighbor of c is h, which is saturated, go to step 6
- Step 6. cha is not M-augmenting

Augmenting Path Algorithm – example

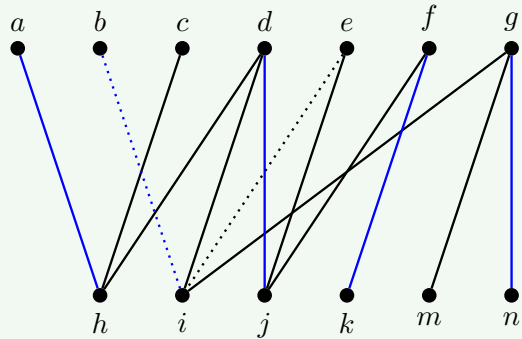
Example



- $U = \{c, e\}$
- Step 3. select $e \in U$
- Step 4. neighbors of e: i, j, consider i
- Step 6. eib

Augmenting Path Algorithm – example

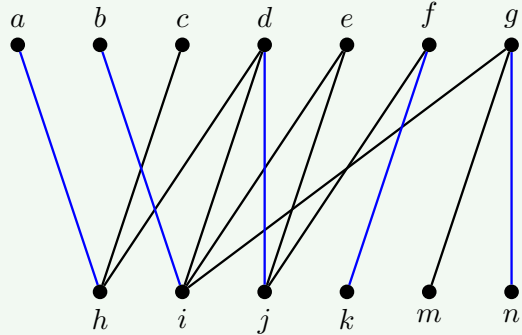
Example



- Step 6. eib is not M-augmenting
- Step 4. neighbors of e: i, j, consider j
- Step 6. $ejdha$, $ejdib$

Augmenting Path Algorithm – example

Example



- Step 6. $ejdha$, $ejdib$, neither is M-augmenting
- No M-augmenting path exists, we have obtained a maximum matching

Vertex cover

Definition

A *vertex cover* Q for a graph G is a subset of vertices so that every edge of G has at least one endpoint in Q.

- Every graph has a vertex cover, e.g. take Q to be the set of all vertices
- But, we would like to optimize the vertex cover, i.e. find a minimum vertex cover

König–Egerváry Theorem

Theorem

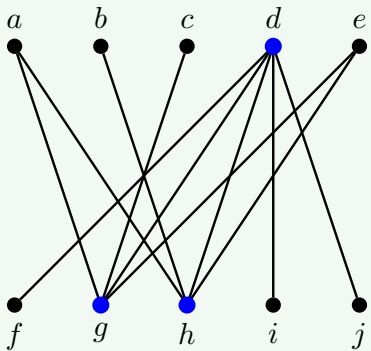
For a bipartite graph G, the size of a maximum matching of G equals the size of a minimum vertex cover for G.

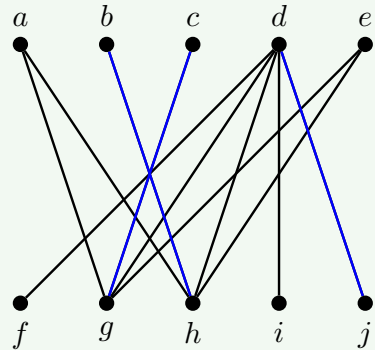
- Given any vertex in the cover, at most one matched edge can be incident to it

König–Egerváry Theorem

Example

- We have discussed that the matching is maximum
- One vertex cover with 3 vertices is $\{g, h, d\}$





Vertex Cover Method

1. Let $G = (X \cup Y, E)$ be a bipartite graph
2. Apply the Augmenting Path Algorithm and mark the vertices considered throughout its final iteration of steps 2-6
3. Define a vertex cover Q as the unmarked vertices from X and the marked vertices from Y
4. Q is a minimum vertex cover for G

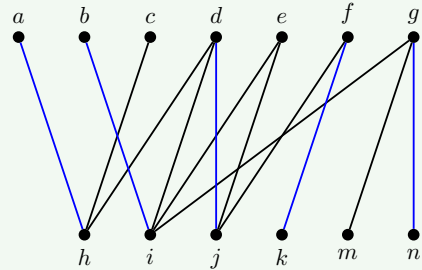
Note

- Step 2 includes not just the vertices in U from Augmenting Path Algorithm, but also any vertex that was reached through an alternating path that originated at a vertex from U.
- Thus the unmarked vertices will be those that are never mentioned during the final step of the implementation of the Augmenting Path Algorithm

Augmenting Path Algorithm final iteration – example

Example

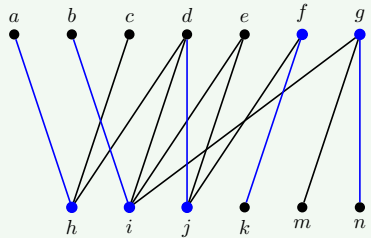
- Step 2. $U = \{c, e\}$
- Step 3. select $c \in U$
- Step 4. only neighbor of c is h, which is saturated, go to step 6
- Step 6. cha is not M-augmenting
- Step 3. select $e \in U$
- Step 4. neighbors of e: i, j, consider i
- Step 6. eib is not M-augmenting
- Step 4. neighbors of e: i, j, consider j
- Step 6. $ejdha$, $ejdib$ are not M-augmenting
- Mark vertices: c, e, h, a, i, j, b, d

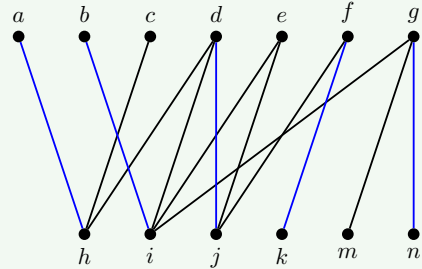


Vertex Cover Method – example

Example

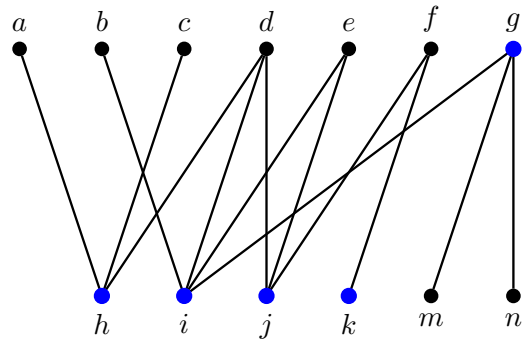
- Mark vertices: c, e, h, a, i, j, b, d
- Unmarked vertices in X: f, g
- Marked vertices in Y:h, i, j
- Size of vertex cover: 5





Remark

- More than one minimum vertex cover may exist for the same graph, just as more than one maximum matching may exist
- In the previous example, we found one such vertex cover through the matching found using the Augmenting Path Algorithm
- Another minimum vertex cover



Remark

- Given a maximum matching, apply Augmenting Path Algorithm starting from step 2 gives a minimum vertex cover
- If we have an X-matching, treat X as Y and Y as X in the Vertex Cover Method.
- If perfect matching, then either X or Y gives a minimum vertex cover because size of minimum vertex cover is equal to size of maximum matching
- We will see more examples in Tutorial 12

Network Flows, Trees, and Matchings

- Flow and capacity
- Rooted trees
- Bipartite graphs
- Matching terminology and strategies
- Augmenting paths and vertex covers
- Stable marriages
- Unacceptable partners

Stable Marriage Problem

- Our terminology will reflect the marriage model
- We start with two distinct, yet equal sized, groups of people, usually men and women, who have ranked the members of the other group
- A matching is *stable* if there are no two people who would both prefer each other to their current partners. That is, there is no unmatched pair that would rather "run away together."

Stable matching

Definition

Let $G = K_{n,n} = (X \cup Y, E)$ be a complete bipartite graph, where each vertex has a strict preference ordering over its neighbors, and let M be a perfect matching in G.

- A pair of unmatched vertices $(x, y) \in X \times Y$ is *unstable* if
  - x prefers y over their current partner, and
  - y prefers x over their current partner
- M is *stable* if no unmatched pair of vertices is unstable

Unstable matching – example

Example

- Four men and four women are being paired into marriages
- Each person has ranked the members of the opposite sex

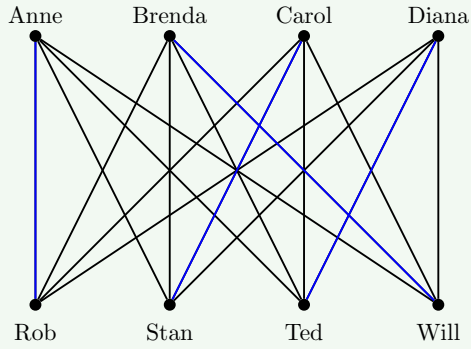
Anne:	$t > r > s > w$	Rob:	$a > b > c > d$
Brenda:	$s > w > r > t$	Stan:	$a > c > b > d$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$r > s > t > w$	Will:	$c > b > a > d$

Unstable matching – example

Example

Anne:	$t > r > s > w$	Rob:	$a > b > c > d$
Brenda:	$s > w > r > t$	Stan:	$a > c > b > d$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$r > s > t > w$	Will:	$c > b > a > d$

- Unstable: Will and Carol prefer each other to their current mate



Gale-Shapley Algorithm

- Proposed by David Gale and Lloyd Shapley
- Designed to find a stable matching between two equally sized groups
- Input: preference rankings of n women and n men
- Output: a stable matching

Remark

The algorithm is asymmetric: the group making the proposals is generally favored.

Gale-Shapley Algorithm – steps

1. Each unmatched man proposes to the highest ranked woman on his list who has not yet rejected him.
2. If every woman receives exactly one proposal, then go to Step 3. Otherwise continue to Step 4.
3. Each woman who receives a proposal accepts it. The resulting matching is stable.
4. Each woman who receives more than one proposal keeps her most preferred proposal as "maybe" and rejects all others.
5. Repeat Steps 1–4 until everyone has been paired.

Gale-Shapley Algorithm – example

Example

Anne:	$t > r > s > w$	Rob:	$a > b > c > d$
Brenda:	$s > w > r > t$	Stan:	$a > c > b > d$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$r > s > t > w$	Will:	$c > b > a > d$

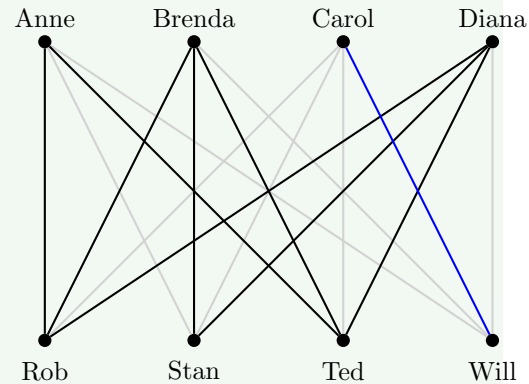
- Step 1. Proposals: Rob-Anne, Stan-Anne, Ted-Carol, Will-Carol
- Step 3. Anne and Carol make choices.
  - Neither Rob nor Stan are Anne's top choice so she rejects the lower ranked one (Stan) and says maybe to the other (Rob)
  - Will is Carol's top choice so she accepts his proposal and rejects Ted

Gale-Shapley Algorithm – example

Example

Rob:	$a > b > c > d$
Stan:	$a > c > b > d$
Ted:	$c > d > a > b$
Will:	$c > b > a > d$

- Step 4. unmatched men propose to the next available woman on their preference list
  - Rob proposes again to Anne since she said "maybe" in the last round.
  - Stan proposes to Brenda; cannot propose to Anne since she rejected him previously
  - Ted proposes to Diana



Gale-Shapley Algorithm – example

Example

Anne:	$t > r > s > w$	Rob:	$a > b > c > d$
Brenda:	$s > w > r > t$	Stan:	$a > c > b > d$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$r > s > t > w$	Will:	$c > b > a > d$

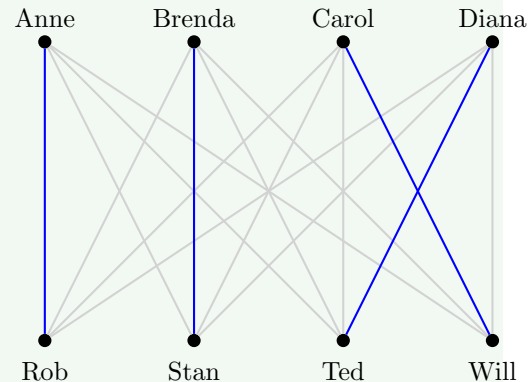
- Step 4. Proposals: Rob-Anne, Stan-Brenda, Ted-Diana
- Step 2. Since all the proposals are different (no woman receives more than one proposal), the women all accept.

Gale-Shapley Algorithm – example

Example

Anne:	$t > r > s > w$
Brenda:	$s > w > r > t$
Carol:	$w > r > s > t$
Diana:	$r > s > t > w$

Rob:	$a > b > c > d$
Stan:	$a > c > b > d$
Ted:	$c > d > a > b$
Will:	$c > b > a > d$



- Two men are paired with their top choice, one with his second, and one with his third
- The same holds for the women (two first choices, one second, and one third)

Gale-Shapley Algorithm

- The Gale–Shapley Algorithm is asymmetric and in fact favors the group making the proposals.
- In the example above
  - Two men are paired with their top choice, one with his second, and one with his third
  - The same holds for the women (two first choices, one second, and one third)
- If the women were the ones proposing we would likely see an improvement in their overall happiness

Gale-Shapley Algorithm – example

Example

Anne:	$t > r > s > w$	Rob:	$a > b > c > d$
Brenda:	$s > w > r > t$	Stan:	$a > c > b > d$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$r > s > t > w$	Will:	$c > b > a > d$

- Step 1. Proposals: Anne-Ted, Brenda-Stan, Carol-Will, Diana-Rob
- Step 2. Since all the proposals are different (no man received more than one proposal), the men must all accept
- The women were all paired with their first choice, whereas only one man was paired with his first choice, two with their third choice, and one with his fourth choice

Remarks

- The group proposing in the Gale-Shapley Algorithm is more likely to be happy
- In the previous example, even though the second matching seems more imbalanced than the first one, it is still a stable matching—there is no guarantee that a unique stable matching exists.
- In fact, many examples have more than one stable matching possible.
- The important concept to remember is that for a complete bipartite graph with rankings, a stable matching will always exist.
- If we generalize this to other types of graphs, the same may not hold.

Network Flows, Trees, and Matchings

- Flow and capacity
- Rooted trees
- Bipartite graphs
- Matching terminology and strategies
- Augmenting paths and vertex covers
- Stable marriages
- Unacceptable partners

Unacceptable partner

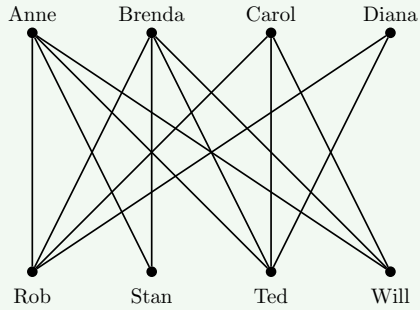
- By having each person rank all others of the opposite sex, we assume that all of these potential matches are acceptable
- This is not realistic in a real-world scenario—some people should never be matched, even if they are the only pair left.
- To adjust for this, we introduce the notion of an unacceptable partner

Unacceptable partner – example

Example

Anne:	$t > r > w$	Rob:	$a > b > c > d$
Brenda:	$w > r > t$	Stan:	$a > b$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$s > r > t$	Will:	$c > b > a$

- If a person is missing from the ranking, then they are deemed unacceptable
- Will is unacceptable to Diana and only Anne and Brenda are acceptable to Stan
- We consider men proposing, so leave out edges to unacceptable partners for men only



Gale-Shapley Algorithm (with Unacceptable Partners)

- We still model the situation using a bipartite graph, but now the graph need not be complete
- Some people may regard certain partners as unacceptable, so not every pair is allowed
- As a result, a stable matching may leave some people unmatched
- A matching is *stable* if there is no unmatched pair x and y such that
  - x and y are acceptable to each other, and
  - each is either unmatched or prefers the other to their current partner
- Input: preference rankings of n women and n men
- Output: a stable matching

Gale-Shapley Algorithm (with Unacceptable Partners) – steps

1. Each unmatched man proposes to the highest ranked acceptable woman on his list who has not rejected him.
2. If every woman receives at most one acceptable proposal, each such proposal is accepted and the resulting matching is stable. Otherwise go to Step 3.
3. Each woman who receives proposals
   i. rejects any proposal from an unacceptable man;
   ii. keeps her most preferred acceptable proposal as "maybe" and rejects the rest.
4. Any rejected man proposes again to the highest ranked acceptable woman on his list who has not rejected him.
5. Repeat Steps 2–4 until everyone is matched or until no unmatched man has any acceptable woman left to propose to.

Remark

- The algorithm always produces a stable matching.
- Some people may remain unmatched.
- The procedure can also be run with women proposing.

Gale-Shapley Algorithm (with Unacceptable Partners) – example

Example

Anne:	$t > r > w$	Rob:	$a > b > c > d$
Brenda:	$w > r > t$	Stan:	$a > b$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$s > r > t$	Will:	$c > b > a$

- Step 1. The initial proposals are Rob –Anne, Stan –Anne, Ted –Carol, and Will – Carol.
- Step 3. Anne and Carol make choices
  - Stan is unacceptable to Anne, so she rejects him; since Rob is not her top choice, she says "maybe".
  - Will is Carol's top choice so she accepts his proposal and rejects Ted

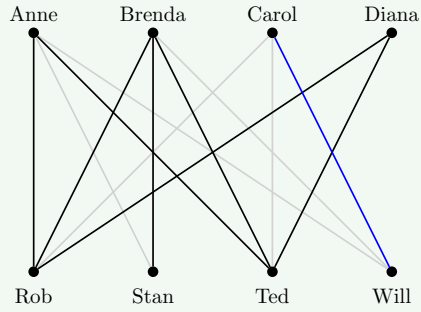
Gale-Shapley Algorithm (with Unacceptable Partners) – example

Example

Anne:	$t > r > w$	Rob:	$a > b > c > d$
Brenda:	$w > r > t$	Stan:	$a > b$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$s > r > t$	Will:	$c > b > a$

Step 4.

- Remaining men propose to the next available woman on their preference list
- Rob - Anne
- Stan - Brenda
- Ted - Diana



Gale-Shapley Algorithm (with Unacceptable Partners) – example

Example

Anne:	$t > r > w$	Rob:	$a > b > c > d$
Brenda:	$w > r > t$	Stan:	$a > b$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$s > r > t$	Will:	$c > b > a$

- Step 4. Rob - Anne, Stan - Brenda, Ted - Diana
- Step 3. Even though all proposals are different, Brenda rejects Stan since he is an unacceptable partner. The other two women say maybe since their proposals are not from their top choice of the available men

Gale-Shapley Algorithm (with Unacceptable Partners) – example

Example

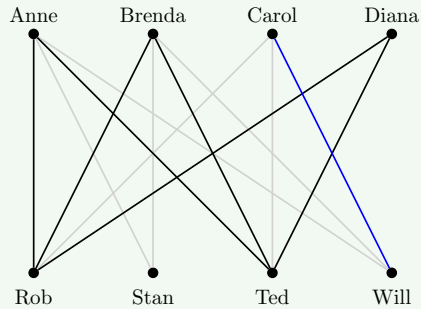
Anne:	$t > r > w$	Rob:	$a > b > c > d$
Brenda:	$w > r > t$	Stan:	$a > b$
Carol:	$w > r > s > t$	Ted:	$c > d > a > b$
Diana:	$s > r > t$	Will:	$c > b > a$

Step 4.

- Rob - Anne
- Ted - Diana
- Stan does not have any acceptable partners left so must remain single

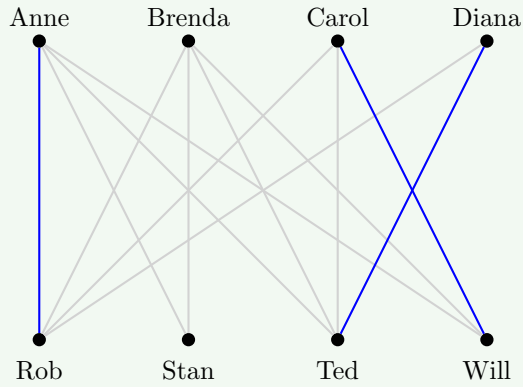
Step 2.

- Both proposals are different and from acceptable partners
- Both women accept



Gale-Shapley Algorithm (with Unacceptable Partners) – example

Example



In a scenario with unacceptable partners, a stable matching can exist without pairing all people.